QORVO SCHOLARSHIP AND TRAINING PROGRAM

SPECTIFICATION

hms_timer

Author: Pham Truong Giang
Instructor: Nguyen Huu Quang

Ha Noi, 21/09/2026

MỤC LỤC

1. Design Overview....1
1.1. Tổng quan chức năng....1
1.2. Block Diagram....1
1.3. I/O Table....2
2. Architecture....3
2.1. Detail block diagram....3
2.2. Instance Table....3
3. Detailed Block Description....5
3.1. Block: input_cond (Input Conditioning)....5
3.1.1. Tổng quan....5
3.1.2. I/O Table....5
3.1.3. Logic hoạt động chi tiết....6
3.2. Block: sel_fsm (Select Field FSM)....8
3.2.1. Tổng quan....8
3.2.2. I/O Table....8
3.2.3. Logic hoạt động chi tiết....9
3.3. Block: freq_divider (Frequency Divider)....10
3.3.1. Tổng quan....10
3.3.2. I/O Table....10
3.3.3. Logic hoạt động chi tiết....10
3.4. Block: time_counter (Time Counter + Adjust)....11
3.4.1. Tổng quan....11
3.4.2. I/O Table....12
3.4.3. Logic hoạt động chi tiết....12

DANH MỤC HÌNH VẼ

Hình 1.1: Block Diagram khối hms_timer (khối top level).....1
Hình 2.1: Mô tả kiến trúc tổng quan cho module hms_timer 3
Hình 3.1: Block diagram module input_cond.....5
Hình 3.2: Mô tả chi tiết logic hoạt động của module input_cond.....6
Hình 3.3: Block diagram modue sel_fsm.....8
Hình 3.4: State diagram cho hoạt động của module sel_fms.....9
Hình 3.5: Block diagram module freq_divider 10
Hình 3.6: Block diagram module time_counter.....11
Hình 3.7: Mô tả chi tiết logic hoạt động bên trong khối time_counter 14
Hình 3.8: Lưu đồ thuật toán phần counter và wrap logic cho second.....15

DANH MỤC BẢNG BIỂU

Bảng 1.1: Ttổng quan tín hiệu input và output khối hms_timer.....2
Bảng 2.1: Tên instance ứng với các module con trong project.....3
Bảng 3.1: Tổng quan tín hiệu input và output khối input_cond.....5
Bảng 3.2: Tổng quan input/output cho tầng 1.....6
Bảng 3.3: Tổng quan input/output cho tầng debounce.....6
Bảng 3.4: Tổng quan input/output cho tầng Rising Edge Detector.....7
Bảng 3.5: Tổng quan input/output module sel_fsm.....8
Bảng 3.6: Tổng quan input/output module freq_divider.....10
Bảng 3.7: Dữ liệu tính toán tham số.....10
Bảng 3.8: Tổng quan input/output của module time_counter.....12
Bảng 3.9: Bảng tín hiệu nội bộ dùng trong module time_counter.....12
Bảng 3.10: Ví dụ về hành vi hoạt động khi đếm tăng.....16
Bảng 3.11: Ví dụ về hành vi hoạt động khi đếm giảm.....16

1. Design Overview

1.1. Tổng quan chức năng

Module **hms_timer** là bộ đếm thời gian thực hiển thị giờ - phút - giây, hoạt động ở tần số clock 1 MHz với reset bất đồng bộ tích cực mức thấp.

Module thực hiện hai chức năng chính:

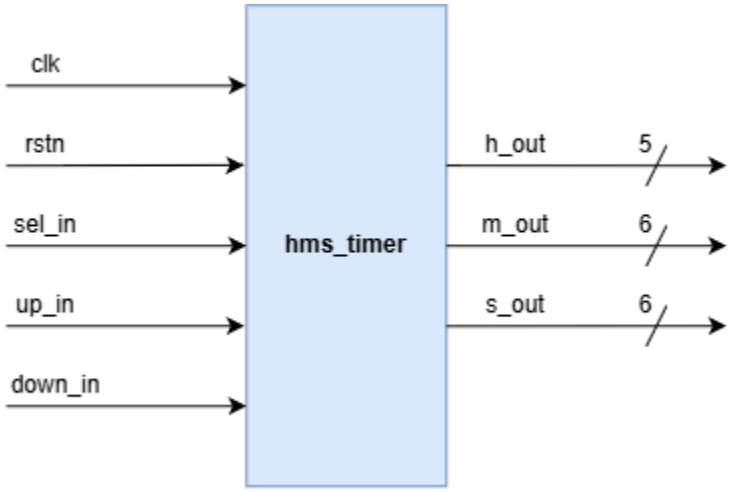
Free-run counting: Tự động đếm giây, phút, giờ theo thời gian thực. Mỗi giây, giá trị second tăng 1. Khi second đạt 59, nó quay về 0 và nhớ lên giá trị minute. Tương tự, minute nhớ lên hour. Hour wrap tại 23 về 0.

Time adjustment: Cho phép người dùng chỉnh giá trị thời gian thông qua 3 nút bấm. Nút Select chọn trường cần chỉnh: Hour/ Minute/ Second. Nút Up tăng giá trị trường đang chọn thêm 1. Nút Down giảm giá trị trường đang chọn đi 1. Cả 3 nút đều là edge-triggered.

Khi adjust trùng thời điểm với free-run tick trên cùng 1 trường, cả hai được cộng dồn. Khi Up và Down cùng xuất hiện trong 1 chu kỳ clock, Up được ưu tiên. Adjust chỉ wrap trong trường đang chọn, không carry sang trường khác.

Đồng hồ tiếp tục free-run trong khi đang adjust, không tạm dừng.

1.2. Block Diagram



Hình 1.1: Block Diagram module hms_timer (khối top level)

1.3. I/O Table

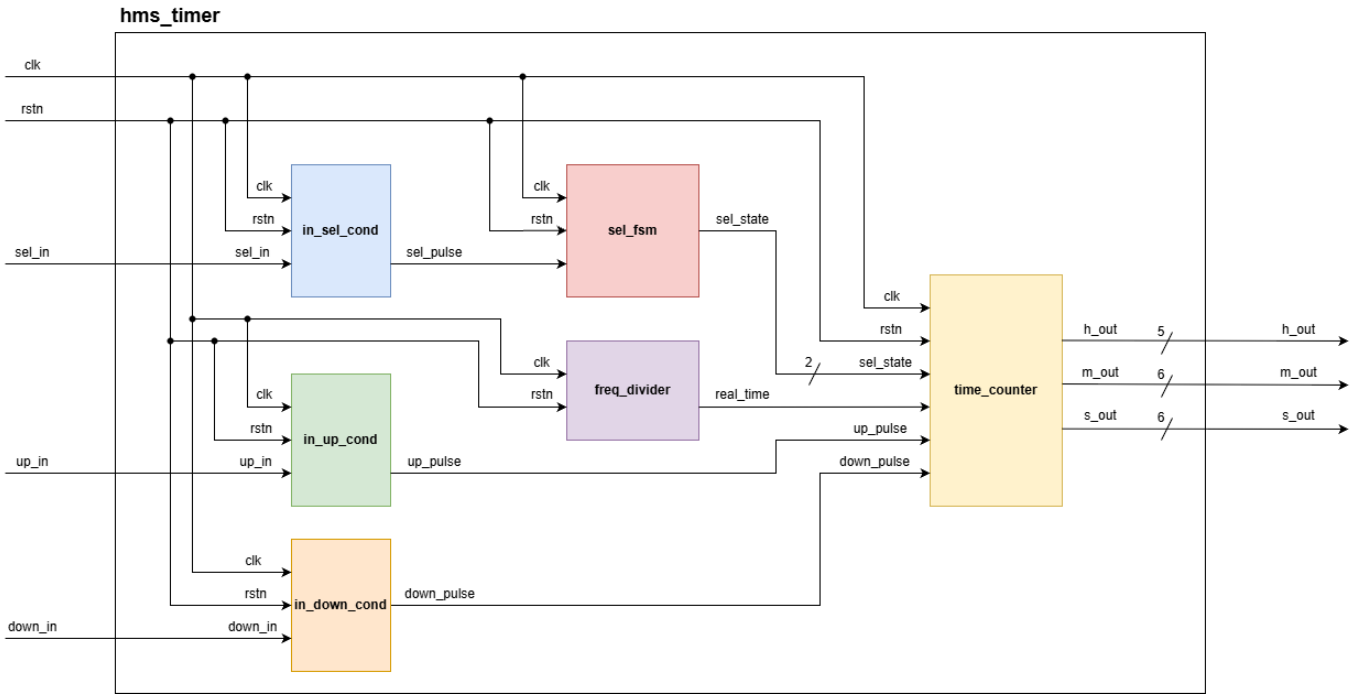
Bảng 1.1: Ttổng quan tín hiệu input và output khối hms_timer

Port	Chiều	Width	Mô tả
clk	Input	1	Tín hiệu clock hệ thống, tần số 1 MHz.
rstn	Input	1	Tín hiệu reset bất đồng bộ, tích cực mức thấp.
sel_in	Input	1	Tín hiệu chọn trường thời gian. Mỗi cạnh lên chuyển trường đang chọn: Hour → Minute → Second → Hour.
up_in	Input	1	Tín hiệu tăng giá trị. Mỗi cạnh lên tăng giá trị trường đang chọn thêm 1.
down_in	Input	1	Tín hiệu giảm giá trị. Mỗi cạnh lên giảm giá trị trường đang chọn đi 1.
h_out	Output	5	Giá trị giờ hiện tại, phạm vi 0–23.
m_out	Output	6	Giá trị phút hiện tại, phạm vi 0–59.
s_out	Output	6	Giá trị giây hiện tại, phạm vi 0–59.

2. Architecture

2.1. Detail block diagram

Module hms_timer gồm 4 sub-module, được instantiate thành 6 instance. Tín hiệu clk và rstn được chia sẻ cho tất cả 6 instance.



Hình 2.1: Tổng quan kiến trúc tổng quan cho module hms_timer

2.2. Instance Table

Bảng 2.1: Tên instance ứng với các module con trong project

Instance Name	Module Name	Mô tả
in_sel_cond	input_cond	Xử lý tín hiệu nút Select: đồng bộ, debounce, tạo pulse
in_up_cond	input_cond	Xử lý tín hiệu nút Up: đồng bộ, debounce, tạo pulse
in_down_cond	input_cond	Xử lý tín hiệu nút Down: đồng bộ, debounce, tạo pulse
sel_fsm	sel_fsm	FSM chọn trường: HOUR/ MIN/ SEC

prescaler	prescaler	Chia tần 1 MHz → 1 Hz
time_counter	time_counter	Đếm giờ/phút/giây + adjust logic

Tổng quan về hoạt động của 4 sub-module:

Module **input_cond** (input conditioning) là khối xử lý tín hiệu đầu vào cho cả 3 nút, tương ứng với 3 instance mà ***Bảng 2.1*** đề cập. Tín hiệu nút bấm bản chất là bất đồng bộ với clock và bị rung cơ học (bouncing) hay nhiễu khi nhấn/nhả. Khối này giải quyết cả hai vấn đề thông qua pipeline 3 tầng:

- *Debounce Counter*: Chỉ công nhận sự thay đổi input khi nó giữ ổn định liên tục một thời gian N nhất định, loại bỏ các cạnh giả do rung động cơ học hoặc nhiễu.
- *2-FF Synchronizer*: Đưa tín hiệu bất đồng bộ vào clock domain, giảm xác suất metastability.
- *Rising Edge Detector*: Chuyển tín hiệu dạng level thành 1 pulse đúng 1 chu kỳ clock tại thời điểm rising edge.

Module **sel_fsm** (Select FSM) là máy trạng thái quản lý trường thời gian đang được chọn để chỉnh. Mỗi khi nhận được sel_pulse, FSM chuyển sang trường tiếp theo.

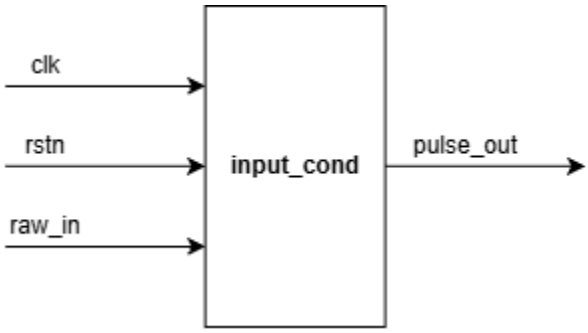
Module **prescaler** (Clock Divider): Bộ chia tần số, chuyển clock 1 MHz thành pulse real_time với tần số 1 Hz.

Module **time_counter** (Time Counter + Adjust) là khối xử lý chính của thiết kế, chứa 3 counter nội bộ cho sec (0–59), min (0–59), hour (0–23). Thực hiện đồng thời hai chức năng là free-run và adjust.

3. Detailed Block Description

3.1. Block: input_cond (Input Conditioning)

3.1.1. Tổng quan



Hình 3.1: Block diagram module input_cond

Module **input_cond** là khối xử lý tín hiệu đầu vào từ nút bấm. Tín hiệu nút bấm có 2 vấn đề cần giải quyết: bất đồng bộ với clock domain gây ra metastability và rung cơ học khi nhấn/nhả bouncing tạo nhiều cạnh giả.

Khối giải quyết cả hai vấn đề và chuyển đổi tín hiệu dạng level thành 1 pulse duy nhất mỗi lần nhấn, thông qua pipeline 3 tầng nối tiếp và được sử dụng cho các instance: *in_sel_cond*, *in_up_cond*, *in_down_cond*. Cả 3 instance có hành vi giống hệt nhau, chỉ khác kết nối I/O.

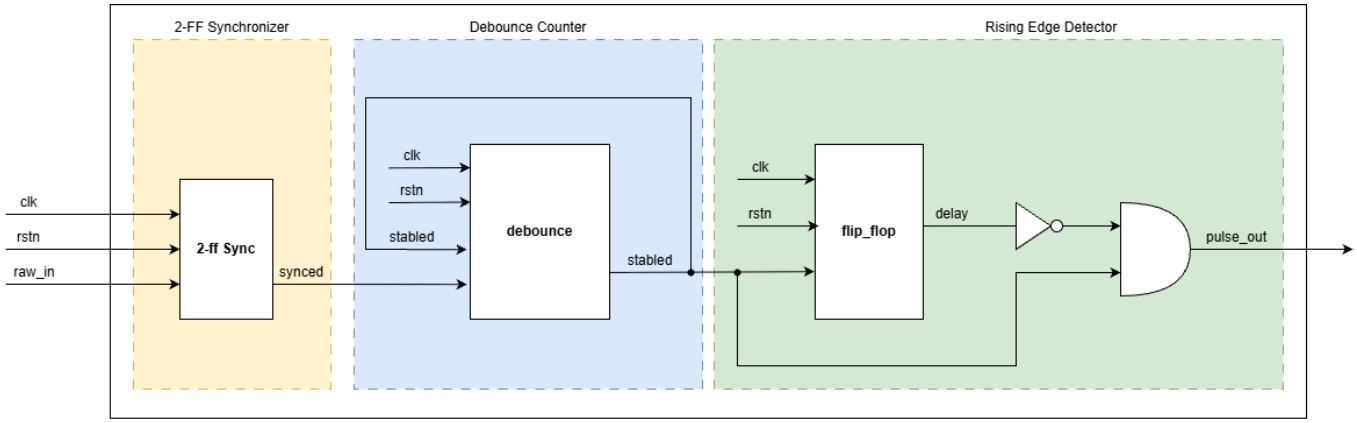
3.1.2. I/O Table

Bảng 3.1: Tổng quan tín hiệu input và output khối input_cond

Port	Chiều	Width	Mô tả
clk	Input	1	Clock hệ thống 1 MHz.
rstn	Input	1	Reset bất đồng bộ, tích cực thấp.
raw_in	Input	1	Tín hiệu từ nút bấm — bất đồng bộ với clock, chưa lọc nhiễu.
pulse_out	Output	1	Pulse đã được biến đổi, high đúng 1 chu kỳ clock tại mỗi rising edge.

3.1.3. Logic hoạt động chi tiết

Để xử lý tín hiệu đầu vào cho cả 3 instance là raw_in từ bất đồng bộ với clock domain chuyển đổi tín hiệu dạng level thành 1 pulse duy nhất mỗi lần nhấn, thông qua pipeline 3 tầng nối tiếp:



Hình 3.2: Mô tả chi tiết logic hoạt động của module input_cond

Tầng 1: 2-FF Synchronizer

Bảng 3.2: Tổng quan input/output cho tầng 1

Tín hiệu	Chiều	Mô tả
clk	Input	Clock hệ thống
rstn	Input	Reset bất đồng bộ, tích cực thấp
raw_in	Input	Tín hiệu thô từ nút bấm, bất đồng bộ
synced	Output	Tín hiệu đã đồng bộ vào clock domain

Sử dụng 2 flip-flop mắc nối tiếp để đưa tín hiệu bất đồng bộ raw_in vào clock domain. FF1 sample trực tiếp raw_in. FF2 sample output của FF1 sau 1 chu kỳ clock chờ ổn định. Output synced phản ánh raw_in với delay 2 chu kỳ clock, đã an toàn trong clock domain.

Tầng 2: Debounce

Bảng 3.3: Tổng quan input/output cho tầng debounce

Tín hiệu	Chiều	Mô tả

clk	Input	Clock hệ thống
rstn	Input	Reset bất đồng bộ, tích cực thấp
synced	Input	Tín hiệu đã đồng bộ từ tầng 1
stabled	Input	Giá trị ổn định hiện tại, đưa quay lại để so sánh với synced
stabled	Output	Giá trị ổn định sau khi debounce

Khối debounce so sánh synced với stabled (hồi tiếp từ chính output) mỗi chu kỳ clock:

Khi synced == stabled: Input chưa thay đổi hoặc đã ổn định. Counter nội bộ reset về 0. stabled giữ nguyên giá trị hiện tại.

Khi synced ≠ stabled: Input đang khác trạng thái ổn định. Counter đếm lên 1 mỗi chu kỳ clock. Nếu trong quá trình đếm mà synced quay lại bằng stabled (do bounce) thì counter bị reset, phải đếm lại từ đầu. Chỉ khi counter đếm đủ N chu kỳ liên tục thì stabled được cập nhật bằng giá trị synced hiện tại.

output: stabled là tín hiệu dạng level (lên 1 và giữ khi nhấn nút, về 0 và giữ khi nhả nút). Mỗi lần thay đổi trạng thái đều bị trễ N chu kì so với synced.

Tầng 3: Rising Edge Detector

Bảng 3.4: Tổng quan input/output cho tầng Rising Edge Detector

Tín hiệu	Chiều	Mô tả
clk	Input	Clock hệ thống
rstn	Input	Reset bất đồng bộ, tích cực thấp.
stabled	Input	Tín hiệu ổn định từ tầng 2.

pulse_out	Output	Pulse 1 chu kỳ clock tại rising edge của stabled

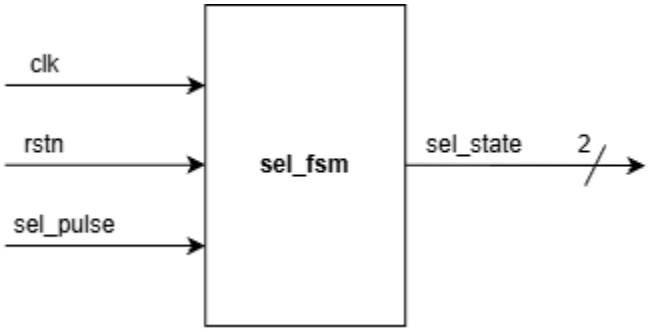
Gồm 3 phần tử logic theo diagram:

- Flip-flop: Lấy mẫu stabled mỗi cạnh lên clock, tạo ra delay là bản sao stabled bị trễ 1 chu kỳ.
- NOT gate: Đảo delay thành ~delay.
- AND gate: Tổ hợp stabled & ~delay tạo pulse_out.

Kết quả: pulse_out chỉ lên mức high khi stabled = 1 và delay = 0 tức đúng 1 cycle tại thời điểm stabled chuyển từ 0 lên 1.

3.2. Block: sel_fsm (Select Field FSM)

3.2.1. Tổng quan



Hình 3.3: Block diagram modue sel_fsm

sel_fsm là máy trạng thái quản lý trường thời gian đang được chọn để chỉnh. Mỗi khi nhận được 1 pulse từ nút Select (sel_pulse), FSM chuyển sang trường tiếp theo theo vòng 1 chiều: HOUR → MIN → SEC → HOUR.

Sau reset, trường mặc định là giờ. Output sel_state[1:0] cho biết trường đang chọn, được khối time_counter sử dụng để xác định nút Up/Down tác động lên trường nào.

3.2.2. I/O Table

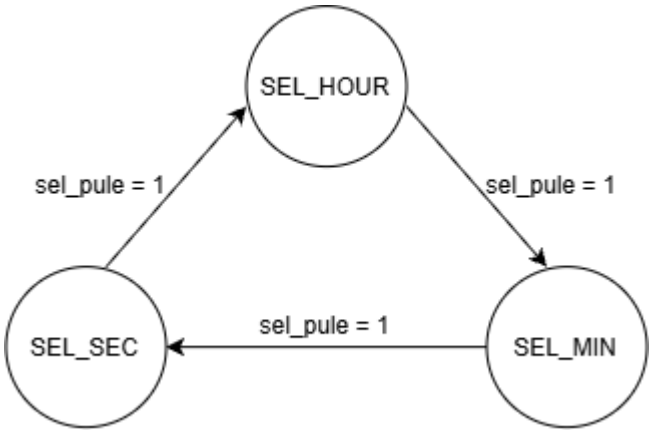
Bảng 3.5: Tổng quan input/output module sel_fsm

Port	Chiều	Width	Mô tả
clk	Input	1	Clock hệ thống 1 MHz

rstn	Input	1	Reset bất đồng bộ, tích cực thấp
sel_pulse	Input	1	Tín hiệu in_sel_cond, mỗi pulse kích hoạt 1 lần chuyển trạng thái.
sel_state	Output	2	Trường đang chọn: 2'b00 = HOUR, 2'b01 = MIN, 2'b10 = SEC

3.2.3. Logic hoạt động chi tiết

FSM có 3 trạng thái hợp lệ, mã hóa 2-bit:



Hình 3.4: State diagram cho hoạt động của module sel_fms

Khối **sel_fsm** chỉ có 1 thanh ghi state với logic chuyển trạng thái như sau:

Reset: Khi rstn = 0, state sẽ mặc định chuyển về SEL_HOUR. Mặc định sau reset luôn chọn trường giờ.

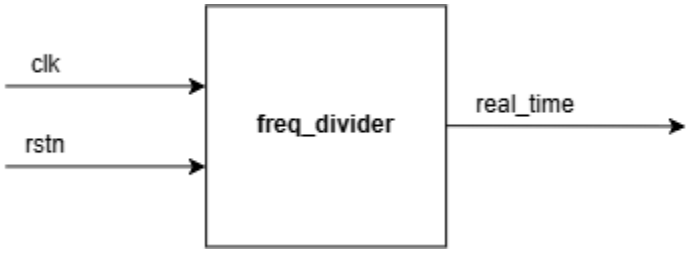
Khi sel_pulse = 0: Không có nhấn nút Select. state giữ nguyên giá trị hiện tại, không thay đổi.

Khi sel_pulse = 1: Nhận được 1 pulse từ nút Select. Chuyển sang trạng thái tiếp theo theo thứ tự lần lượt SEL_HOUR → SEL_MIN → SEL_SEC.

Output: sel_state = state, output là giá trị trực tiếp từ thanh ghi.

3.3. Block: freq_divider (Frequency Divider)

3.3.1. Tổng quan



Hình 3.5: Block diagram module freq_divider

freq_divider là bộ chia tần số, chuyển clock hệ thống 1 MHz thành pulse real_time với tần số 1 Hz. Sử dụng counter nội bộ đếm từ 0 đến 999.999. Mỗi khi counter đạt giá trị max, real_time được assert high trong đúng 1 chu kỳ clock rồi về 0 để tạo nhịp 1s cho khối **time_counter** thực hiện free-run counting.

3.3.2. I/O Table

Bảng 3.6: Tổng quan input/output module freq_divider

Port	Chiều	Width	Mô tả
clk	Input	1	Clock hệ thống 1 MHz
rstn	Input	1	Reset bất đồng bộ, tích cực thấp
real_time	Output	1	Pulse 1 chu kỳ clock mỗi giây. High khi counter đạt max, low ở tất cả các cycle còn lại.

3.3.3. Logic hoạt động chi tiết

3.3.3.1. Tính toán tham số

Mục tiêu: từ clock 1 MHz tạo pulse 1 Hz → cần đếm 1.000.000 chu kỳ cho mỗi giây.

Bảng 3.7: Dữ liệu tính toán tham số

Tham số	Giá trị	Ý nghĩa
Tần số clock	1 MHz	1 chu kỳ = 1 µs
Số cycle cần đếm	1.000.000	Đếm từ 0 đến 999.999

Giá trị max counter	999.999	CLK_FREQ – 1 (một giá trị parameter mặc định là 1.000.000)
Độ rộng counter	20 bit	2^{20} = 1.048.576 > 999.999

3.3.3.2. Logic hoạt động

Khối **freq_divider** có logic gồm 1 counter và 1 comparator:

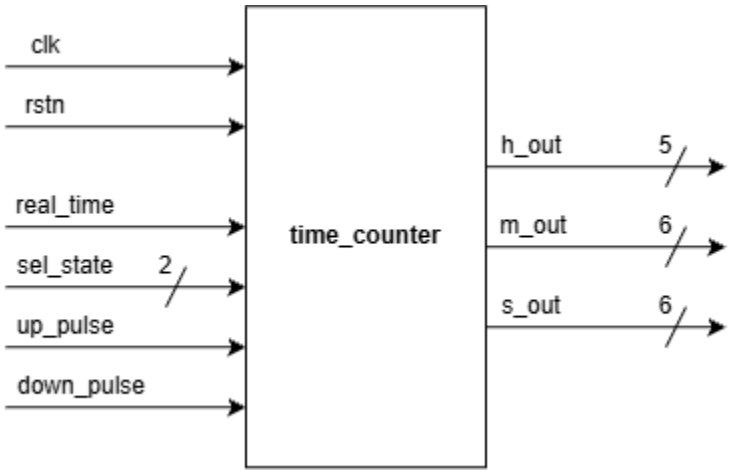
Reset: Khi rstn = 0, count được đặt về 0 và real_time cũng được đặt về 0. Counter về trạng thái ban đầu.

Khi count < CLK_FREQ − 1: Counter chưa đạt max. count tăng thêm 1 mỗi chu kỳ clock. real_time = 0.

Khi count == CLK_FREQ − 1**:** Counter đạt giá trị max (999.999). real_time = 1, assert high đúng 1 chu kỳ clock. count reset về 0 để bắt đầu chu kỳ đếm mới.

3.4. Block: time_counter (Time Counter + Adjust)

3.4.1. Tổng quan



Hình 3.6: Block diagram module time_counter

time_counter là khối xử lý chính của thiết kế, chứa 3 counter nội bộ cho sec (0–59), min (0–59), hour (0–23). Khối thực hiện đồng thời hai chức năng:

Free-run counting: Khi nhận pulse real_time từ freq_divider, tự động tăng sec. Khi sec tràn 59, carry lên min. Khi min tràn 59, carry lên hour. Hour wrap tại 23 về 0.

Time adjustment: Khi nhận up_pulse hoặc down_pulse, tăng/giảm giá trị trường được chỉ định bởi sel_state. Adjust chỉ wrap trong trường đó, không carry sang trường khác.

Khi free-run và adjust trùng cùng 1 chu kỳ clock trên cùng 1 trường, cả hai được cộng dồn. Khi up_pulse và down_pulse cùng xuất hiện, up_pulse được ưu tiên hơn.

3.4.2. I/O Table

Bảng 3.8: Tổng quan input/output của module time_counter

Port	Chiều	Width	Mô tả
clk	Input	1	Clock hệ thống 1 MHz
rstn	Input	1	Reset bất đồng bộ, tích cực thấp
real_time	Input	1	Pulse 1 Hz từ freq_divider
up_pulse	Input	1	Pulse tăng giá trị trường đang chọn
down_pulse	Input	1	Pulse giảm giá trị trường đang chọn
sel_state	Input	2	Trường đang chọn: 00 = HOUR, 01 = MIN, 10 = SEC
h_out	Output	5	Giá trị giờ hiện tại (0–23)
m_out	Output	6	Giá trị phút hiện tại (0–59)
s_out	Output	6	Giá trị giây hiện tại (0–59)

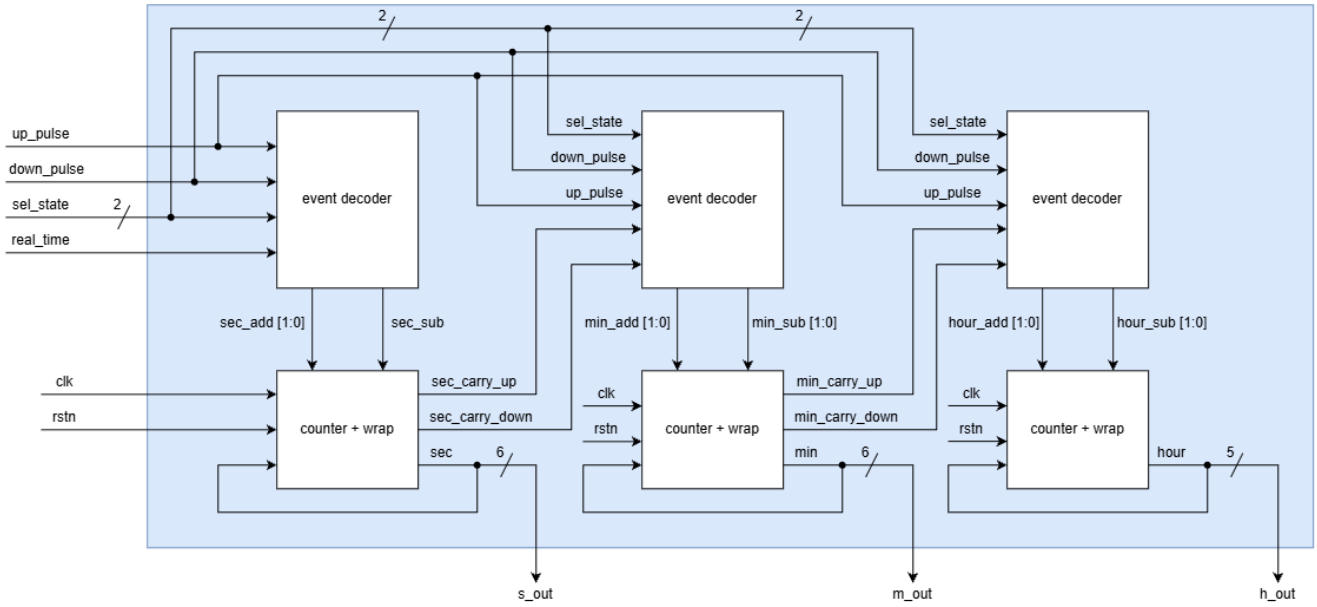
3.4.3. Logic hoạt động chi tiết

Bảng 3.9: Bảng tín hiệu nội bộ dùng trong module time_counter

Signal	Width	Mô tả
sec	6	Thanh ghi giây (0–59)
min	6	Thanh ghi phút (0–59)
hour	5	Thanh ghi giờ (0–23)
sec_inc	1	Sự kiện free-run: chính là giá trị real_time
sec_up	1	Sự kiện điều chỉnh tăng sec

sec_dn	1	Sự kiện điều chỉnh giảm sec
sec_add	2	Tổng phần tăng sec (0–2)
sec_sub	1	Tổng phần giảm sec (0–1)
sec_carry_up	1	Nhớ lên min khi sec tràn trên
sec_carry_down	1	Mượn từ min khi sec tràn dưới
min_up	1	Sự kiện điều chỉnh tăng min
min_dn	1	Sự kiện điều chỉnh giảm min
min_add	2	Tổng phần tăng min (carry + adjust)
min_sub	2	Tổng phần giảm min (borrow + adjust)
min_carry_up	1	Nhớ lên hour khi min tràn trên
min_carry_down	1	Mượn từ hour khi min tràn dưới
hour_up	1	Sự kiện điều chỉnh tăng hour
hour_dn	1	Sự kiện điều chỉnh giảm hour
hour_add	2	Tổng phần tăng hour (carry + adjust)
hour_sub	2	Tổng phần giảm hour (borrow + adjust)

Logic time_counter gồm 3 tầng xử lý nối tiếp: SEC → MIN → HOUR. Mỗi tầng có cấu trúc giống nhau gồm 2 phần: Event Decoder và Counter + Wrap Logic.



Hình 3.7: Mô tả chi tiết logic hoạt động bên trong khối time_counter

3.4.3.1. Logic cho sec

Phần Event Decoder

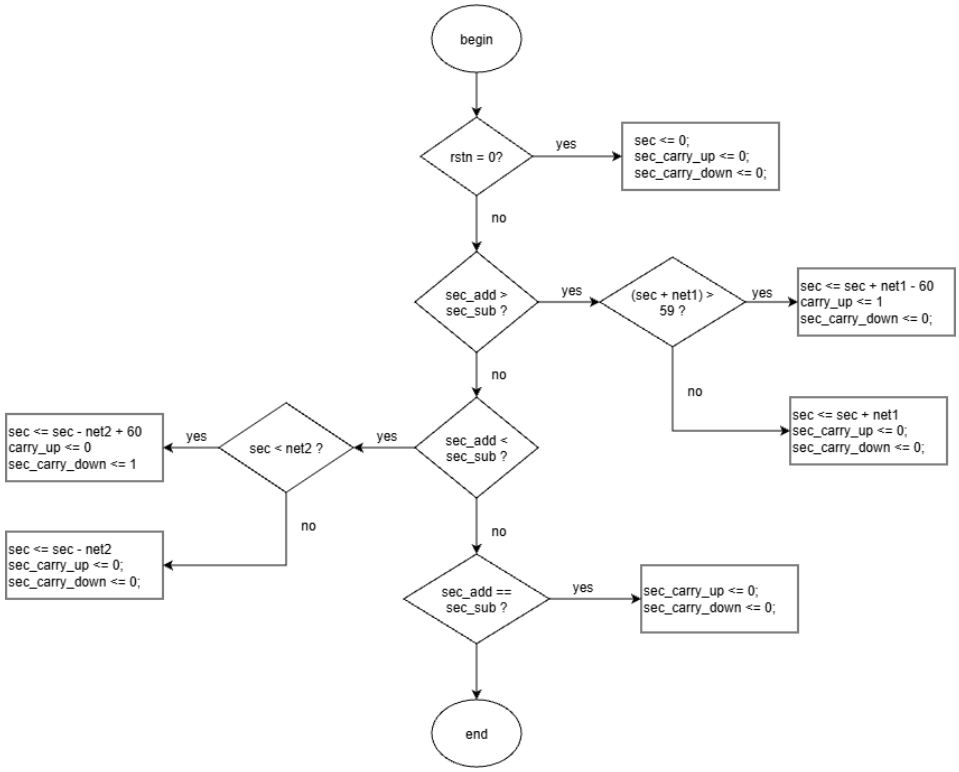
Các xung từ ngoài gửi vào down_pulse và up_pulse cộng với trạng thái của tín hiệu sel_state sẽ được đi qua logic tổ hợp để tạo ra giá trị đếm lên và đếm xuống, output của các tín hiệu này sẽ được dùng để xác định 2 tín hiệu cho phần Counter + Wrap là sec_add và sec_sub như biểu diễn logic ở dưới

```
sec_inc = real_time
sec_up  = (sel_state == 2'b10)  &&  up_pulse
sec_dn  = (sel_state == 2'b10)  &&  down_pulse  &&  !up_pulse

sec_add [1:0] = sec_inc + sec_up
sec_sub   = sec_dn
```

Phần Counter + Wrap

Logic phần này được thực hiện theo lưu đồ thuật toán sau:



Hình 3.8: Lưu đồ thuật toán phần counter và wrap logic cho second

Trong lưu đồ ***Hình 3.8*** mô tả logic xử lý trong 1 chu kỳ clock, thực thi tại mỗi posedge clk. Gồm 1 nhánh reset và 3 nhánh xử lý chính dựa trên quan hệ giữa sec_add và sec_sub.

Hai biến trung gian được sử dụng:

- net1 = sec_add − sec_sub : là biến dùng để lưu kết quả khi tăng
- net2 = sec_sub − sec_add : là biến dùng để lưu kết quả khi giảm

Nếu (sec_add > sec_sub) biểu thị cho trường hợp nhất nút tăng tại second. Tiếp tục kiểm tra xem có tràn trên hay không ((sec + net1) > 59?)

- Nếu có điều kiện tràn trên, vượt quá 59 thì sec <= sec + net1 – 60 và nhớ sang cho min
- Nếu không thấy tràn trên thì đếm bình thường

Bảng 3.10: Ví dụ về hành vi hoạt động khi đếm tăng

sec	net1	sec + net1	> 59?	sec (mới)	Sec_carry_up
23	1	24	No	24	0
59	1	60	Yes	0	1
59	2	61	Yes	1	1
58	2	60	Yes	0	1

Nếu sec_add < sec_sub biểu thị cho trường hợp nhấn nút giảm đồng thời không nhấn nút tăng tại nút second. Tiếp tục kiểm tra tràn dưới: sec < net2?

- Nếu có điều kiện tràn dưới (sec < 1) thì sec <= sec − net2 + 60 và nhớ cờ nhớ sec_carry_down sang cho min
- Nếu không có điều kiện tràn dưới thì trừ và đếm bình thường

Bảng 3.11: Ví dụ về hành vi hoạt động khi đếm giảm

sec	net2	sec < net2?	sec mới	Sec_carry_down
30	1	No	29	0
1	1	No	0	0
0	1	Yes	59	1

Nếu sec_add == sec_sub (sảy ra khi real_time = 1 đồng thời down_pulse = 1 trên SEC) khi đó sec vẫn giữ nguyên và các cờ nhớ bị set về 0